INSTITUTO UNIVERSITARIO DE TECNOLOGÍA
JUAN PABLO PÉREZ ALFONZO
IUTEPAL

# MANUAL TÉCNICO Y DEL SISTEMA

## Repositorio Digital y Gestión Documental (TEG)

Arquitectura, Procesamiento Documental, Seguridad Anti-Plagio y Operaciones

**Versión del Sistema:** 2.2.0 (Edición Integral con Anti-Plagio)
**Fecha de Publicación:** Octubre 2026
**Institución:** IUTEPAL - Extensión Puerto Cabello
**Destinatarios:** Ingenieros de Software, Analistas y Administradores TI

## 1 FICHA DE TRANSFERENCIA Y VISIÓN GENERAL

Este manual proporciona una radiografía técnica de la arquitectura, flujos internos, esquemas de persistencia y protocolos operacionales del Repositorio Documental del IUTEPAL. El sistema centraliza, cataloga, audita y protege la producción científica institucional: Trabajos Especiales de Grado (TEG), Hojas de Resumen Académico y Recursos Multimedia.

### Capacidades y Características Principales (v2.2.0)

**1. Autonomía 100% Offline:** Ejecución autónoma en redes de campus o servidores locales mediante base de datos **SQLite embebida** ( `data/database.sqlite` ), con conmutación instantánea a **MySQL/MariaDB**.

**2. Conversión Automática Word a PDF:** Pipeline transparente que admite archivos `.pdf` , `.docx` y `.doc` , convirtiendo los documentos de Word a PDF de alta resolución mediante MS Word COM o LibreOffice.

**3. Seguridad Anti-Plagio y Propiedad Intelectual:** Estampado de marca de agua diagonal semitransparente ( `IUTEPAL PUERTO CABELLO` ) y cifrado de permisos PDF con `pypdf` para bloquear copia de texto ( `Ctrl+C` ), edición o extracción de páginas.

**4. CRUD Dinámico de Carreras:** Gestión completa de especialidades en BD ( `carreras` ), parametrizando nombres, códigos, colores hexadecimales e iconos visuales (acceso rápido vía clic derecho para administradores).

**5. Control de Acceso Basado en Roles (RBAC Hardened):** Ocultamiento estricto a nivel de CSS/DOM ( `.admin-only-btn` ) para garantizar que las herramientas de administración jamás sean visibles para estudiantes o visitantes.

## 2 ARQUITECTURA DE SOFTWARE Y TOPOLOGÍA

El sistema opera bajo un modelo desacoplado cliente-servidor estructurado en capas:



### 2.1. Ciclo de Vida de una Petición de Carga (/upload y /addTrabajo)

1. **Recepción Multi-Formato:** El cliente envía un formulario multipart mediante `FormData` con un archivo `.pdf`, `.docx` o `.doc`.
2. **Almacenamiento Temporal Multer:** Multer recibe el archivo en disco en la carpeta `uploads/` con un nombre saneado y timestamp único.
3. **Disparo del Procesador de Seguridad:** `server.js` invoca a `processDocument(tempFilePath)` en `scripts/documentProcessor.js`.
4. **Ejecución del Pipeline Python (process_document.py):**
   • Si la extensión es Word, `docx2pdf` genera el PDF nativo equivalente.
   • Con `reportlab`, sintetiza en memoria la marca de agua *IUTEPAL PUERTO CABELLO* y la fusiona en cada hoja.
   • Con `pypdf`, cifra el PDF con clave de propietario aleatoria, dejando lectura libre pero restringiendo la copia de texto (`PRINT | PRINT_TO_REPRESENTATION`).
5. **Persistencia de Metadatos:** El cliente envía la petición POST `/addTrabajo` con título, carrera, año, tipo y la ruta `/uploads/archivo.pdf`.
6. **Inserción Unificada:** `db.execute()` guarda el registro en la base de datos activa y retorna confirmación.

## 3 ESTRUCTURA DEL PROYECTO Y SCRIPTS DE MANTENIMIENTO

Componentes principales de la arquitectura interna de directorios del proyecto:

| Archivo / Ruta | Categoría | Descripción y Función Técnica |
|---|---|---|
| `scripts/process_document.py` | Pipeline Python | Transforma Word a PDF, estampación vectorial de marca de agua y cifrado de permisos anti-plagio. |
| `scripts/documentProcessor.js` | Orquestador Node.js | Wrapper asíncrono para la invocación del subproceso Python durante la carga de archivos. |
| `scripts/watermark_existing_uploads.py` | Mantenimiento CLI | Procesador por lotes que aplicó la protección de marca de agua y cifrado al lote histórico en `uploads/`. |
| `scripts/migrate.js` | Utilidad CLI | Migrador bi-direccional entre almacenamiento en la nube Cloudinary y almacenamiento en disco local. |
| `scripts/seed.js` | Inicializador BD | Crea las tablas en SQLite/MySQL y siembra los datos iniciales de carreras, usuarios y trabajos de grado. |

## 4 MATRIZ EXHAUSTIVA DE VARIABLES DE ENTORNO (.ENV)

| Variable | Tipo | Valores Permitidos | Propósito Técnico |
|---|---|---|---|
| `PORT` | Numérico | `3000`, `8080` | Puerto TCP de escucha del servidor HTTP Express. |
| `STORAGE_TYPE` | String | `'local'`, `'cloudinary'` | Define el destino de subida de archivos (disco o nube). |
| `UPLOAD_DIR` | String | `uploads` | Carpeta física local para el almacenamiento persistente. |
| `DB_CLIENT` | String | `'sqlite'`, `'mysql'` | Motor de persistencia a inicializar por `db.js`. |
| `SQLITE_DB_PATH` | Ruta | `/data/database.sqlite` | Ubicación del archivo de base de datos SQLite embebida. |
| `DB_HOST` | Host | `localhost`, IP | Dirección de red del servidor MySQL / MariaDB. |
| `DB_PORT` | Numérico | `3306` | Puerto de conexión del servicio MySQL. |
| `DB_USER` | String | `root`, usuario | Usuario con permisos sobre la base de datos MySQL. |
| `DB_PASSWORD` | String | `""`, clave | Contraseña de autenticación para la base de datos. |
| `DB_NAME` | String | `document_upload` | Nombre de la base de datos relacional del sistema. |
| `MYSQL_CONNECTION_LIMIT` | Numérico | `3`, `5` | Límite máximo de conexiones concurrentes en el pool. |

## 5 DICCIONARIO DE DATOS Y MODELO RELACIONAL

### Tabla: carreras

- `id` (INT, PK, AI): Identificador de la carrera.
- `nombre` (VARCHAR(100), NOT NULL): Nombre oficial.
- `codigo` (VARCHAR(50), UNIQUE): Código (ej: INFORMATICA).
- `color` (VARCHAR(20)): Color hexadecimal (ej: #00bcd4).
- `icono` (TEXT): Ruta relativa a la imagen/icono.
- `creado_en` (TIMESTAMP): Marca de tiempo de registro.

### Tabla: usuario

- `id` (INT, PK): ID 0 = Administrador Maestro.
- `nombre` (VARCHAR(100), NOT NULL): Nombre completo.
- `email` (VARCHAR(100), UNIQUE): Correo de acceso.
- `password` (VARCHAR(255)): Contraseña del usuario.
- `esIutepalista` (TINYINT): 1 = Estudiante, 0 = Visitante.

### Tabla: trabajos

- `id` (VARCHAR(50), PK): ID temporal ('t'+timestamp).
- `nombre` (TEXT, NOT NULL): Título formal del TEG.
- `carrera` (VARCHAR(50)): Código de la carrera asociable.
- `ano` (INT): Año académico de la defensa.
- `tipo` (VARCHAR(20)): trabajo, resumen, presentacion, video.
- `pdf_url` (TEXT): Ruta local al PDF protegido.

### Tabla: logs

- `id` (INT, PK, AI): Identificador único del evento.
- `usuario_id` (INT, FK -> usuario.id): Usuario ejecutor.
- `action` (VARCHAR(50)): Evento (access, view, download).
- `doc_id` (VARCHAR(50), FK -> trabajos.id): Documento.
- `fecha` (VARCHAR(100)): Estampa de fecha y hora.

## 6 CATÁLOGO DE ENDPOINTS DE LA API RESTFUL (SERVER.JS)

| Método | Endpoint | Acceso / Rol | Propósito y Operación |
|---|---|---|---|
| `POST` | `/upload` | Administrador | Recibe archivo via Multer y procesa la seguridad anti-plagio con Python. |
| `GET` | `/trabajos` | Público | Retorna el catálogo completo de investigaciones con filtros opcionales. |
| `POST` | `/addTrabajo` | Administrador | Registra los metadatos de un nuevo trabajo en la base de datos. |
| `PUT` | `/updateTrabajo/:id` | Administrador | Actualiza título, carrera, año o tipo de una investigación. |
| `DELETE` | `/deleteTrabajo/:id` | Administrador | Elimina la ficha en la BD y borra el PDF físico en `uploads/`. |
| `GET` | `/carreras` | Público | Obtiene el catálogo de especialidades ofertadas con sus colores e iconos. |
| `POST` | `/login-admin` | Administrador | Autenticación de personal técnico/administrativo por usuario y clave. |
| `POST` | `/login-guest` | Estudiante / Visita | Acceso por correo electrónico, validando estatus de estudiante IUTEPAL. |
| `POST` | `/log-action` | Autenticado | Registra eventos de auditoría (descargas, vistas) en la tabla `logs`. |
| `GET` | `/report` | Administrador | Genera reporte de métricas del repositorio (total trabajos, desgloses). |

## 7 PROTOCOLO DE RESOLUCIÓN DE INCIDENCIAS (RUNBOOK)

**Incidencia 1: Error EADDRINUSE: address already in use :::3000**

Proceso previo reteniendo el puerto 3000. Liberar en Windows via PowerShell/CMD:

```
npx kill-port 3000
# O manualmente mediante consulta de PID:
netstat -ano | findstr :3000  =>  taskkill /PID <PID_ENCONTRADO> /F
```

**Incidencia 2: Error al convertir documento de Word a PDF (docx2pdf)**

Microsoft Word no instalado en el servidor o el servicio COM no tiene permisos de ejecución interactiva.
• Verifique Word en host Windows o asegure contar con `libreoffice-writer` en Linux.
• Compruebe la librería Python ejecutando: `python -c "import docx2pdf; print('OK')"`

**Incidencia 3: Error EBUSY: resource busy or locked al eliminar documento**

Un visor local de PDF (ej: Adobe Acrobat) tiene bloqueado el archivo en el host impidiendo que `fs.unlinkSync` lo elimine. Cierre el visor externo. Los navegadores web no bloquean el archivo de forma persistente.

**Incidencia 4: Los botones de administración parpadean o aparecen en modo usuario**

Caché de navegador activa. Verifique que `menu.html` cargue recursos con el parámetro de versión `?v=3` y fuerce la recarga limpia con `Ctrl + F5`. La regla `body:not(.is-admin) .admin-only-btn { display: none !important; }` garantiza la invisibilidad física en el DOM.

## 8 COMANDOS NPM DE GESTIÓN Y DESPLIEGUE

| Comando | Acción Técnica |
|---|---|
| `npm start` | Inicia el servidor Node.js Express en modo producción ( `http://localhost:3000` ). |
| `npm run dev` | Ejecuta el servidor en modo desarrollo con reinicio automático (nodemon). |
| `npm run seed` | Puebla la base de datos con las carreras, usuarios y trabajos iniciales. |
| `npm run migrate` | Sincroniza y descarga archivos desde Cloudinary al disco local en `uploads/` . |
| `npm run generate-pdf` | Compila y genera de forma automatizada el manual del analista en PDF. |

## 9 RECOMENDACIONES DE SEGURIDAD Y BUENAS PRÁCTICAS

**1. Hashing Criptográfico de Contraseñas:** Migrar campos de clave en texto plano hacia funciones hash seguras ( `bcrypt` o `argon2id` ) con factor de trabajo adecuado.
**2. Tokens de Sesión Seguros:** Sustituir la persistencia en `localStorage` por Cookies `HttpOnly` , `SameSite Strict` y `Secure` firmadas con JWT.
**3. Respaldo Automático de Datos:** Configurar una tarea programada (Cron en Linux o Task Scheduler en Windows) para respaldar semanalmente la carpeta `uploads/` y la base de datos `data/database.sqlite` .